

Лабораторна робота №7. Створення вебзастосунка використовуючи фреймворк Flask: реалізація аутентифікації користувача, панелі адміністратора, структурована організація проєкту.

Завдання. До проєкту лабораторної роботи №6 додати режим аутентифікації користувача, для адміністратора реалізувати «панель адміністратора» з функціоналом повного доступу до даних та акаунтів користувачів – базовий рівень складності, та надсилання листів – для розгорнутого рівня реалізації. Організувати структуру на налаштування проєкт на основі класів – базовий рівень, та ескізів Blueprint та Flask Application Factory – для розгорнутого.

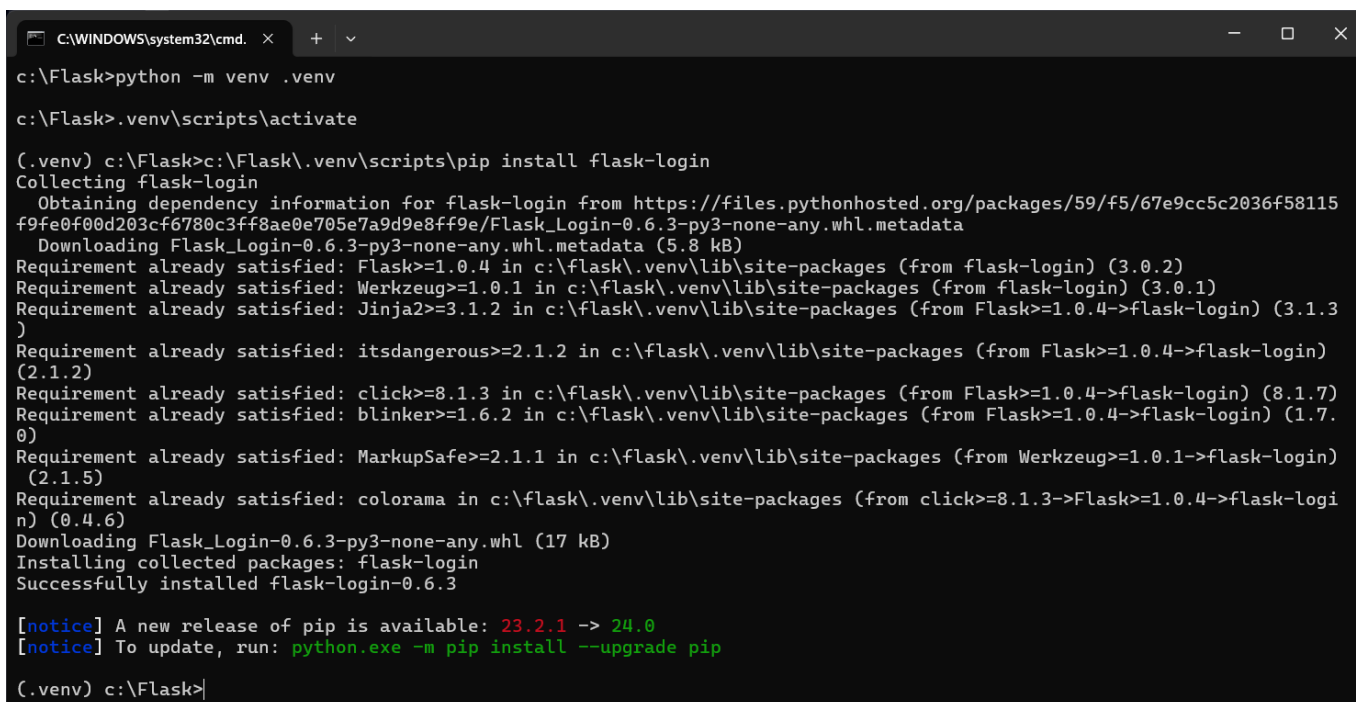
ЗМІСТ

1 АУТЕНТИФІКАЦІЯ У FLASK.....	3
1.1 Створення моделі користувача	3
1.2 Хешування паролів	5
1.3 Інтеграція Flask-Login.....	8
1.4 Обмеження доступу до перегляду	10
1.5 Створення форми авторизації	14
1.6 Авторизація користувачів	14
1.7 Завершення сеансів користувачів (вихід з акаунтів).....	18
1.8 Фінальні штрихи.....	20
2 СТРУКТУРА ТА ЕСКІЗ ДОДАТКА FLASK	20
2.1 Налаштування на основі класів	21
2.2 Створення пакета додатка	23
2.3 Порядок виконання	30
2.4 Запуск проєкту.....	31

1 АУТЕНТИФІКАЦІЯ У FLASK

Аутентифікація - один із найважливіших елементів веб-додатків. Цей процес запобігає потраплянню неавторизованих користувачів на непризначені для них сторінки. Власну систему аутентифікації можна створити за допомогою кукі та хешування паролів. Такий мініатюрний проект стане чудовою перевіркою отриманих навичок.

Як можна було здогадатися, вже існує розширення, яке може значно полегшити життя. Flask-Login - це розширення, що дає змогу легко інтегрувати систему автентифікації в додаток Flask. Встановити його можна за допомогою такої команди:



```

C:\WINDOWS\system32\cmd. x + v
c:\Flask>python -m venv .venv
c:\Flask>.venv\scripts\activate
(.venv) c:\Flask>c:\Flask\.venv\scripts\pip install flask-login
Collecting flask-login
  Obtaining dependency information for flask-login from https://files.pythonhosted.org/packages/59/f5/67e9cc5c2036f58115f9fe0f00d203cf6780c3ff8ae0e705e7a9d9e8ff9e/Flask_Login-0.6.3-py3-none-any.whl.metadata
  Downloading Flask_Login-0.6.3-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: Flask>=1.0.4 in c:\flask\.venv\lib\site-packages (from flask-login) (3.0.2)
Requirement already satisfied: Werkzeug>=1.0.1 in c:\flask\.venv\lib\site-packages (from flask-login) (3.0.1)
Requirement already satisfied: Jinja2>=3.1.2 in c:\flask\.venv\lib\site-packages (from Flask>=1.0.4->flask-login) (3.1.3)
Requirement already satisfied: itsdangerous>=2.1.2 in c:\flask\.venv\lib\site-packages (from Flask>=1.0.4->flask-login) (2.1.2)
Requirement already satisfied: click>=8.1.3 in c:\flask\.venv\lib\site-packages (from Flask>=1.0.4->flask-login) (8.1.7)
Requirement already satisfied: blinker>=1.6.2 in c:\flask\.venv\lib\site-packages (from Flask>=1.0.4->flask-login) (1.7.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in c:\flask\.venv\lib\site-packages (from Werkzeug>=1.0.1->flask-login) (2.1.5)
Requirement already satisfied: colorama in c:\flask\.venv\lib\site-packages (from click>=8.1.3->Flask>=1.0.4->flask-login) (0.4.6)
Downloading Flask_Login-0.6.3-py3-none-any.whl (17 kB)
Installing collected packages: flask-login
Successfully installed flask-login-0.6.3

[notice] A new release of pip is available: 23.2.1 -> 24.0
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) c:\Flask>

```

1.1 Створення моделі користувача

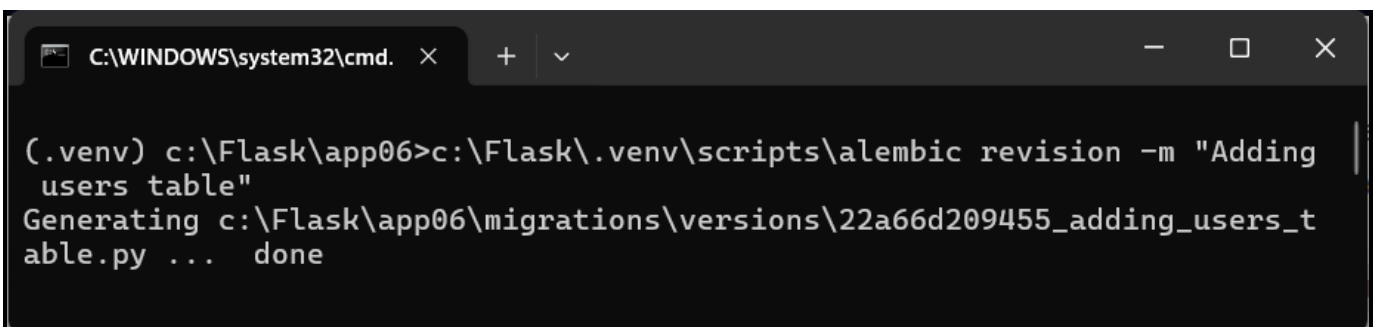
Зараз інформація про користувачів, які є адміністраторами або редакторами сайту, ніде не зберігається. Перше завдання - створити модель *User* для зберігання користувацьких даних. Відкриємо `main.py`, щоб додати модель *User* після моделі *Employee*:

```
# ...
class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(100))
    username = db.Column(db.String(50), nullable=False, unique=True)
    email = db.Column(db.String(100), nullable=False, unique=True)
    password_hash = db.Column(db.String(256), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    updated_on = db.Column(db.DateTime(),
default=datetime.utcnow, onupdate=datetime.utcnow)

    def __repr__(self):
        return "<{}:{}]>".format(self.id, self.username)

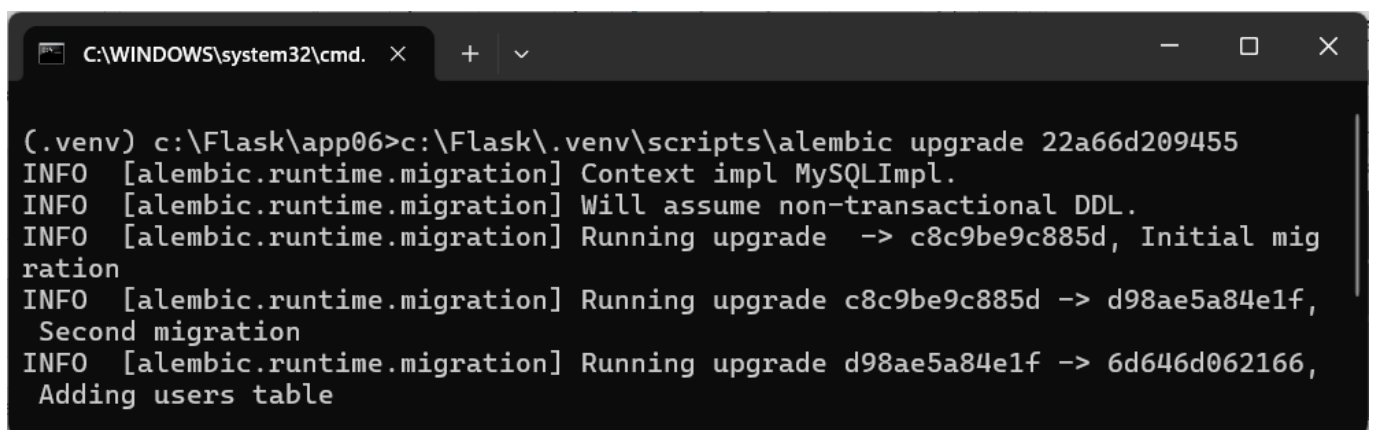
# ...
```

Для оновлення бази даних потрібно створити нову міграцію. У терміналі для створення нового скрипта міграції необхідно ввести таку команду:



```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app06>c:\Flask\.venv\scripts\alembic revision -m "Adding
users table"
Generating c:\Flask\app06\migrations\versions\22a66d209455_adding_users_t
able.py ... done
```

Запустити міграцію необхідно за допомогою команди upgrade:



```
C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app06>c:\Flask\.venv\scripts\alembic upgrade 22a66d209455
INFO [alembic.runtime.migration] Context impl MySQLImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> c8c9be9c885d, Initial mig
ration
INFO [alembic.runtime.migration] Running upgrade c8c9be9c885d -> d98ae5a84e1f,
Second migration
INFO [alembic.runtime.migration] Running upgrade d98ae5a84e1f -> 6d646d062166,
Adding users table
```

Це створить таблицю *users* у базі даних.

1.2 Хешування паролів

Паролі ніколи не повинні зберігатися у вигляді чистого тексту в базі даних. Якщо так робити, зловмисник, здатний зламати базу даних, отримає можливість дізнатися і паролі, і електронні адреси. Відомо, що люди використовують один і той самий пароль для різних сайтів, а це означає, що одна комбінація відкриє зловмисникові доступ до решти акаунтів користувачів.

Замість зберігання паролів прямо в базі даних, потрібно зберігати їхні хеші. Хеш - це рядок символів, які виглядають так, нібито були підібрані випадково.

```
pbkdf2:sha256:50000$0tfe3YgZ$4fc9f1d2de2b6beb0b888278f21a8c0777e8ff980016e043f3eacea9f48f6dea
```

Хеш створюється за допомогою односторонньої функції хешування. Вона приймає довжину змінної і повертає виведення фіксованої довжини, яку ми і називаємо хешем. Безпечним хеш робить той факт, що його не можна використовувати для отримання початкового рядка (тому функція і називається односторонньою). Проте для одного введення одностороння функція хешування односторонньо повертатиме один і той самий результат.

Ось процеси, які задіяні під час створення хеша пароля:

Коли користувач передає пароль (на етапі реєстрації), необхідно його хеширувати і зберегти хеш у базу даних. Коли користувач буде знову авторизуватися, функція повторно створить хеш і порівняє його з тим, що зберігається в базі даних. Якщо вони збігаються, користувач отримає доступ до акаунту. В іншому випадку, виникне помилка.

Flask постачається з пакетом *Werkzeug*, у якому є дві допоміжні функції для хешування паролів.

Метод	Опис
<code>generate_password_hash(password)</code>	Приймає пароль і повертає хеш. За замовчуванням використовує односторонню функцію <code>pbkdf2</code> для створення хешу.
<code>check_password_hash(password_hash, password)</code>	Приймає хеш і пароль у чистому вигляді, потім порівнює <code>password</code> і <code>password_hash</code> . Якщо вони однакові, повертає <code>True</code> .

Наступний код демонструє, як працювати з цими функціями:

```

C:\WINDOWS\system32\cmd. x + v
(.venv) c:\Flask\app06>c:\Flask\.venv\scripts\python
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bi
t (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from werkzeug.security import generate_password_hash, check_password_hash
>>> hash = generate_password_hash("secret password")
>>>
>>> check_password_hash(hash, "secret password")
True
>>> check_password_hash(hash, "pass")
False
>>> |

```

Варто звернути увагу, що коли `check_password_hash()` викликається з правильними паролем ("*secret password*"), повертається `True`, а якщо з неправильними - `False`.

Далі потрібно оновити модель `User`, і додати в неї хешування паролів:

```

# ...
from werkzeug.security import generate_password_hash, check_password_hash
# ...
updated_on = db.Column(db.DateTime(),
default=datetime.utcnow, onupdate=datetime.utcnow)

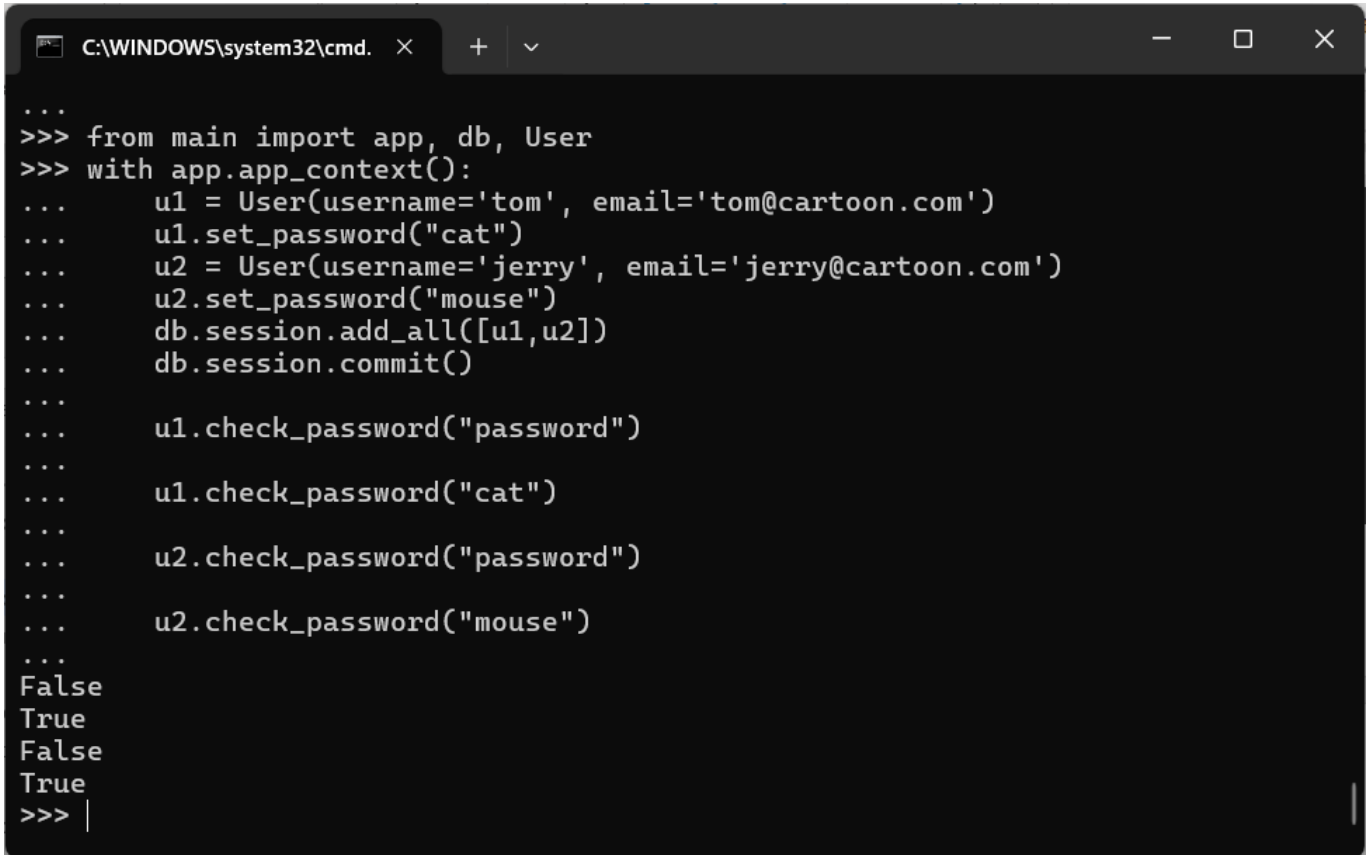
def __repr__(self):
    return "<{}:{}]>".format(self.id, self.username)

def set_password(self, password):
    self.password_hash = generate_password_hash(password)

def check_password(self, password):
    return check_password_hash(self.password_hash, password)
...

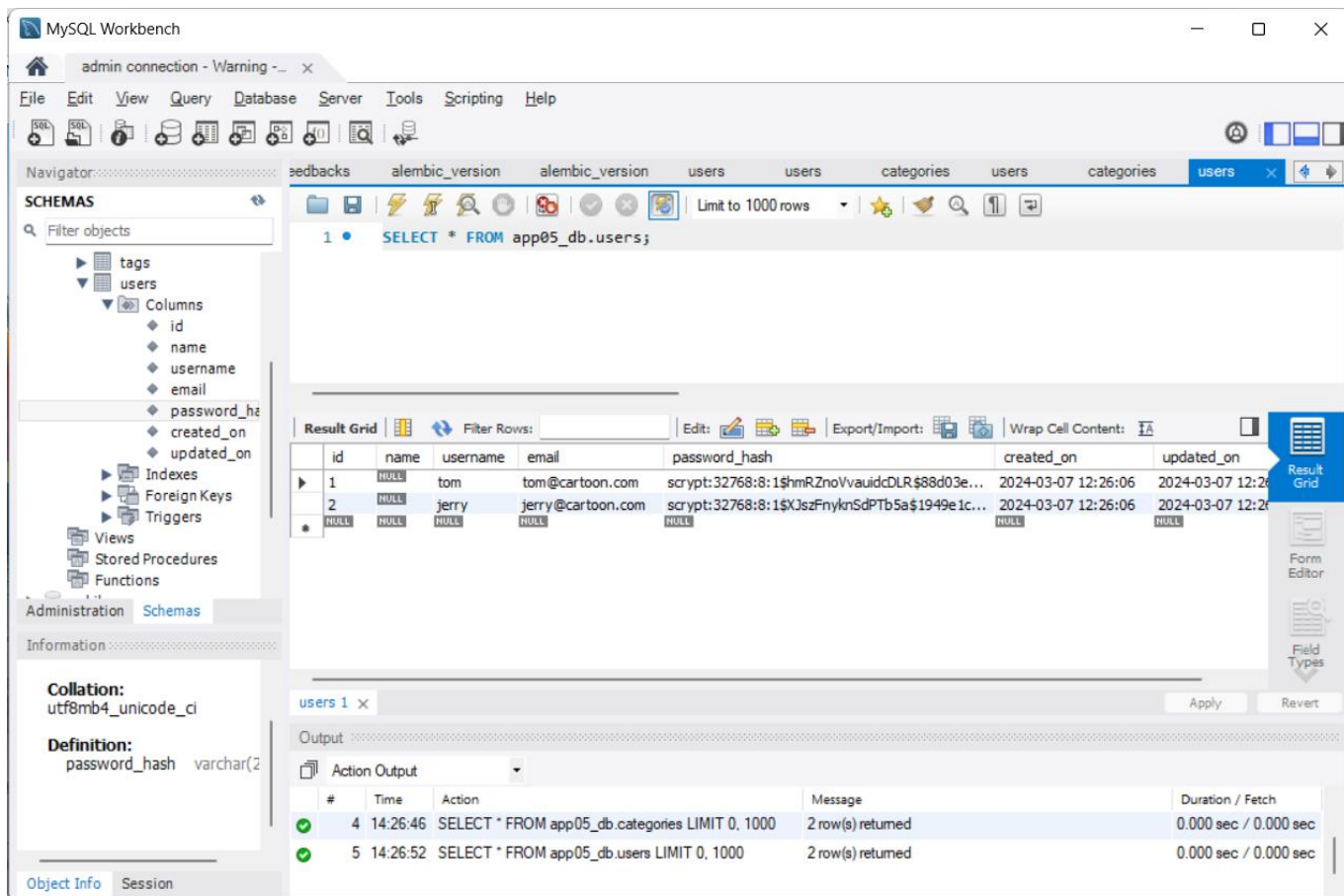
```

Створимо користувачів, щоб перевірити хешування паролів.



```
C:\WINDOWS\system32\cmd. x + v
...
>>> from main import app, db, User
>>> with app.app_context():
...     u1 = User(username='tom', email='tom@cartoon.com')
...     u1.set_password("cat")
...     u2 = User(username='jerry', email='jerry@cartoon.com')
...     u2.set_password("mouse")
...     db.session.add_all([u1,u2])
...     db.session.commit()
...
...     u1.check_password("password")
...
...     u1.check_password("cat")
...
...     u2.check_password("password")
...
...     u2.check_password("mouse")
...
False
True
False
True
>>> |
```

Відображення демонструє нам, що все працює як потрібно, і в базі даних тепер є два користувачі:



1.3 Інтеграція Flask-Login

Для запуску *Flask-Login* потрібно імпортувати клас *LoginManager* з пакета *flask_login* і створити новий екземпляр *LoginManager*:

```
# ...
from werkzeug.security import generate_password_hash, check_password_hash
from flask_login import LoginManager

app = Flask(__name__)

app.config.update(
    DEBUG=True,
    SECRET_KEY='strong secret key',
    SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://admin:masterkey@localhost/app05_db'
)

db = SQLAlchemy(app)
migrate = Migrate(app, db)
login_manager = LoginManager(app)
# ...
```

Для перевірки користувачів Flask-Login вимагає додавання кількох методів у клас `User`. Ці методи перераховані в наступній таблиці:

Метод	Опис
<code>is_authenticated()</code>	Повертає <code>True</code> , якщо користувач перевірений (тобто, зайшов із коректним паролем). В іншому випадку - <code>False</code> .
<code>is_active()</code>	Повертає <code>True</code> , якщо дію акаунта не призупинено.
<code>is_anonymous()</code>	Повертає <code>True</code> для неавторизованих користувачів.
<code>get_id()</code>	Повертає унікальний ідентифікатор об'єкта <code>User</code> .

Flask-Login пропонує реалізацію цих методів за замовчуванням за допомогою класу `UserMixin`. Так, замість визначення їх вручну, можна налаштувати їх успадкування з класу `UserMixin`. Відкриємо `main.py`, щоб змінити заголовок моделі `User`:

```
#...
from flask_login import LoginManager, UserMixin
#...
class User(db.Model, UserMixin):
    __tablename__ = 'users'
#...
```

Залишилося тільки додати зворотний виклик `user_loader`. Відповідний метод можна додати над моделлю `User`.

```
#...
@login_manager.user_loader
def load_user(user_id):
    return db.session.query(User).get(user_id)
#...
```

Функція, що приймає як аргумент декоратор `user_loader`, буде викликатися з кожним запитом до сервера. Вона завантажує користувача з ідентифікатора користувача в куку сесії. `Flask-Login` робить завантаженого користувача доступним за допомогою проксі `current_user`. Для використання `current_user` його потрібно імпортувати з пакета `flask_login`. Він поводить себе як глобальна змінна і доступний як

у функціях подання, так і в шаблонах. У будь-який момент часу *current_user* посилається або на того, хто увійшов у систему, або на анонімного користувача. Розрізнити їх можна за допомогою атрибута *is_authenticated* проксі *current_user*. Для анонімних користувачів *is_authenticated* поверне *False*. В іншому разі - *True*.

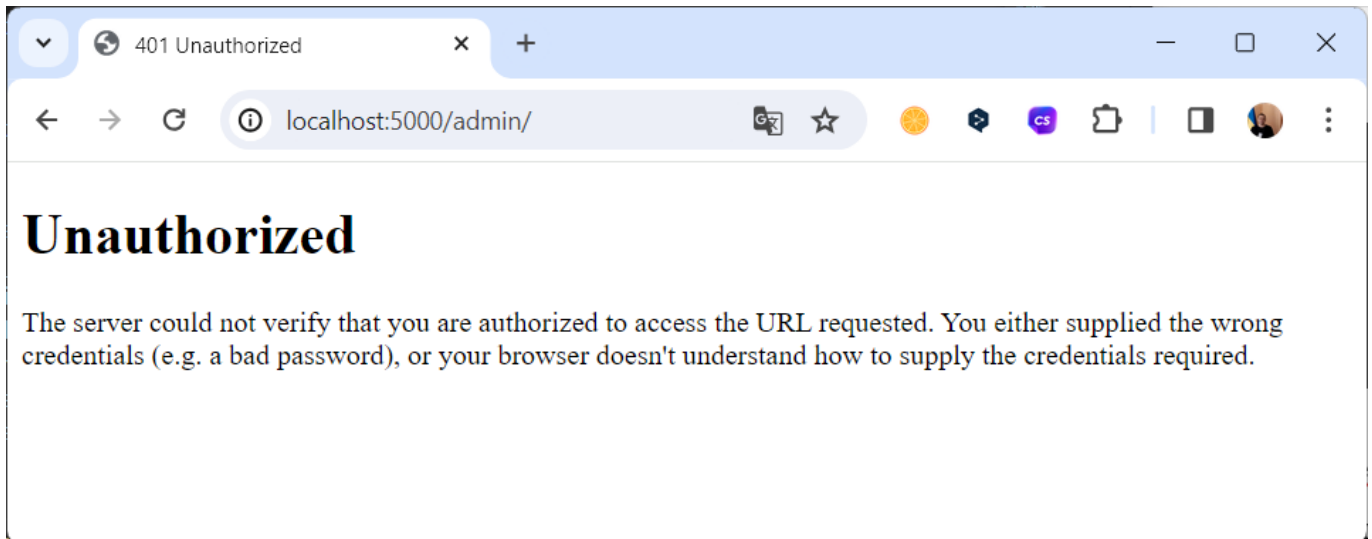
1.4 Обмеження доступу до перегляду

Поки що на сайті немає ніякої адміністративної панелі. У цій темі вона буде представлена звичайною сторінкою. Щоб не допустити неавторизованих користувачів до захищених сторінок у *Flask-Login* є декоратор *login_required*. Додамо наступний код у файлі *main.py* відразу за функцією подання *updating_session()*:

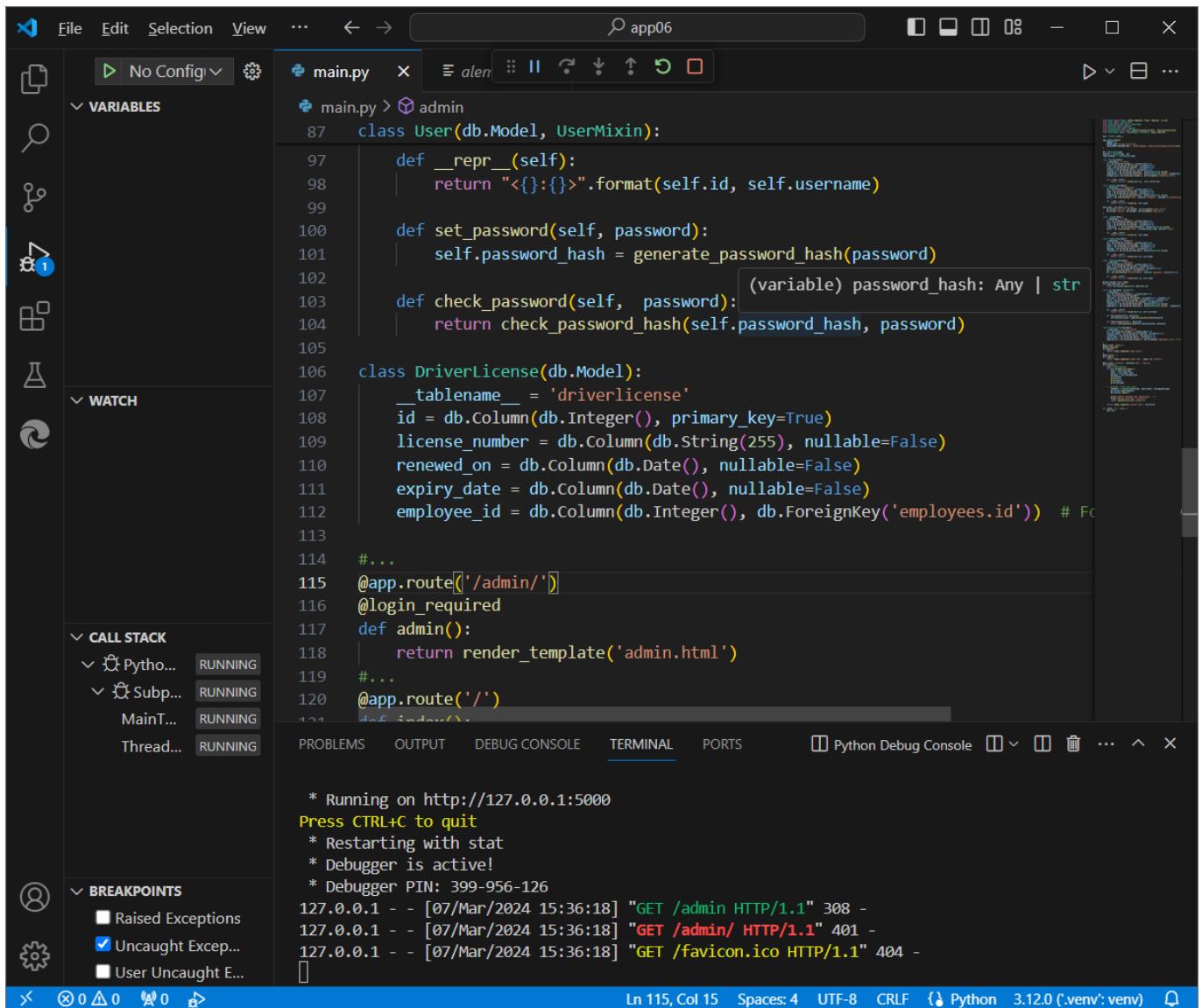
```
#...
from flask_login import LoginManager, UserMixin, login_required
#...
@app.route('/admin/')
@login_required
def admin():
    return render_template('admin.html')
#...
```

Декоратор *login_required* гарантує, що функція подання *admin()* викличеться тільки в тому разі, якщо користувач авторизований. За замовчуванням, якщо анонімний користувач спробує зайти на захищену сторінку, він отримає помилку 401 "Не авторизований".

Необхідно запустити сервер і зайти на <http://localhost:5000/admin>, щоб перевірити, як це працює. Відкриється така сторінка:



Теж саме буде відображено у протоколі роботи сервера:



Замість того щоб показувати користувачеві помилку 401, краще перенаправити його на сторінку авторизації. Щоб зробити це, потрібно передати атрибуту `login_view` екземпляра `LoginManager` значення функції подання `login()`:

```
#...
login_manager = LoginManager(app)
login_manager.login_view = 'login'
#...
```

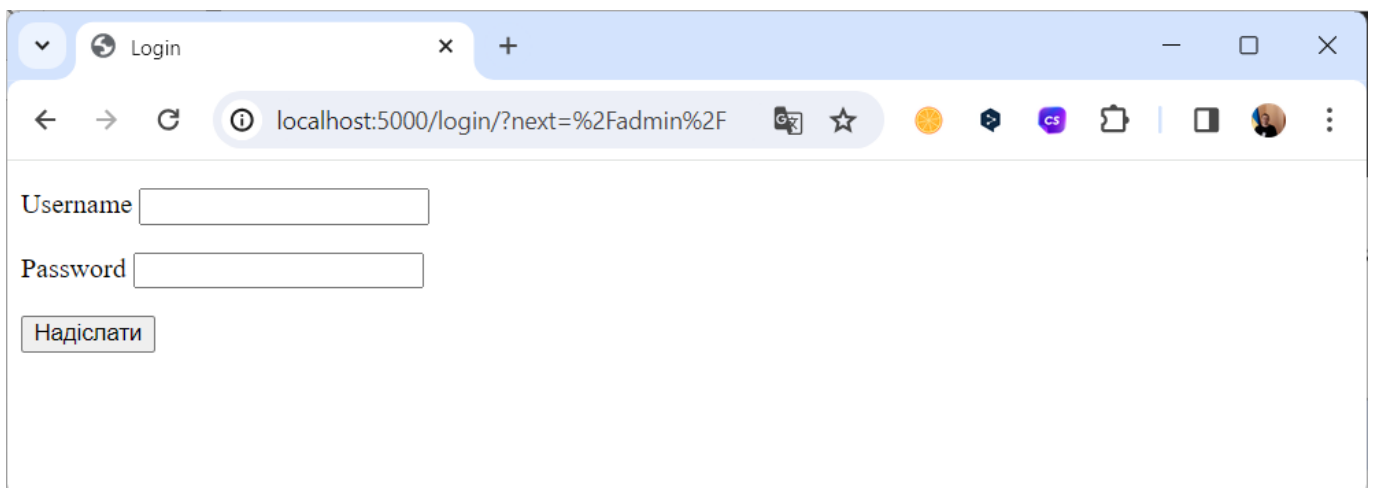
Зараз функція `login()` визначена таким чином (але її потрібно буде поміняти):

```
#...
@app.route('/login/', methods=['post', 'get'])
def login():
    message = ''
    if request.method == 'POST':
        print(request.form)
        username = request.form.get('username')
        password = request.form.get('password')

        if username == 'root' and password == 'pass':
            message = "Correct username and password"
        else:
            message = "Wrong username or password"

    return render_template('login.html', message=message)
#...
```

Якщо тепер зайти на <http://localhost:5000/admin/>, відбудеться перенаправлення на сторінку авторизації:



The screenshot shows a web browser window with the title 'Login'. The address bar displays 'localhost:5000/login/?next=%2Fadmin%2F'. The page content includes a login form with two input fields: 'Username' and 'Password'. Below these fields is a button labeled 'Надіслати' (Submit). The browser's developer tools and various extension icons are visible in the top right corner.

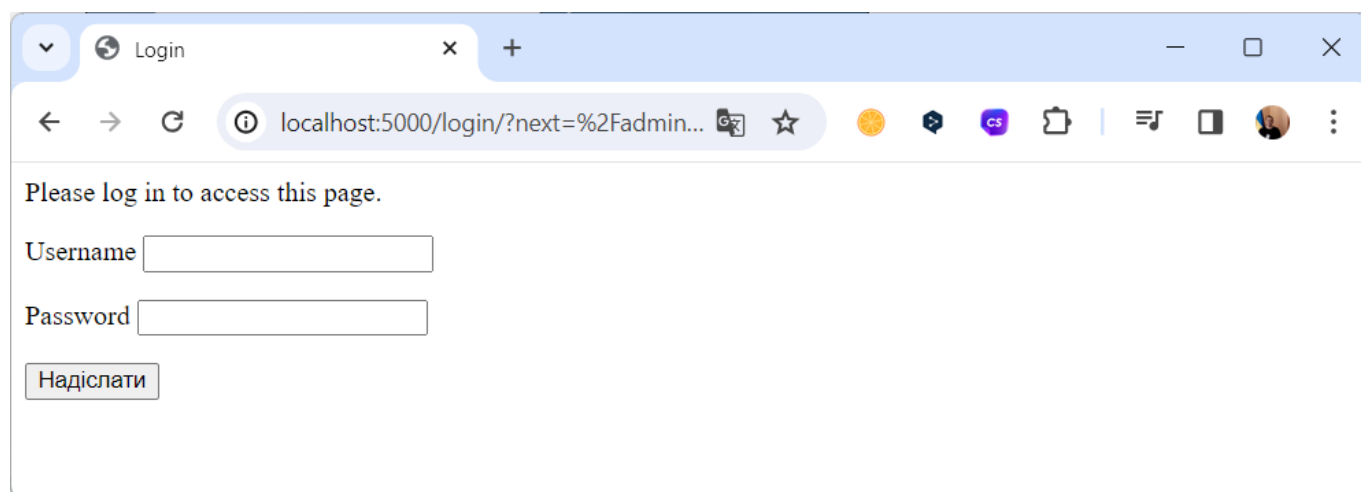
Flask-Login також налаштовує спливаюче повідомлення, коли користувача перенаправляють на сторінку авторизації, але наразі жодного повідомлення немає, тому що шаблон авторизації (*template/login.html*) не відображає жодних повідомлень. Потрібно відкрити *login.html* і додати такий код перед тегом `<form>`:

```
{% endif %}

{% for category, message in get_flashed_messages(with_categories=true) %}
    <spam class="{{ category }}">{{ message }}</spam>
{% endfor %}

<form action="" method="post">
```

Якщо знову зайти на <http://localhost:5000/admin/>, на сторінці відобразиться повідомлення.



Щоб змінити зміст повідомлення, потрібно передати новий текст атрибуту *login_message* екземпляра *LoginManager*.

Заодно чому б не створити шаблон для функції подання *admin()*. Створимо новий шаблон *admin.html* з таким кодом:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
```

```

<body>

<h2>Logged in User details</h2>

<ul>
  <li>Username: {{ current_user.username }}</li>
  <li>Email: {{ current_user.email }}</li>
  <li>Created on: {{ current_user.created_on }}</li>
  <li>Updated on: {{ current_user.updated_on }}</li>
</ul>

</body>
</html>

```

Тут використовується змінна *current_user* для відображення подробиць про авторизованого користувача.

1.5 Створення форми авторизації

Перед авторизацією потрібно створити форму. У ній буде три поля: ім'я користувача, пароль і запам'ятати мене. Відкриємо *forms.py*, щоб додати клас *LoginForm* під класом *ContactForm*:

```

#...
from wtforms import StringField, SubmitField, TextAreaField, BooleanField,
PasswordField
#...
class LoginForm(FlaskForm):
    username = StringField("Username", validators=[DataRequired()])
    password = PasswordField("Password", validators=[DataRequired()])
    remember = BooleanField("Remember Me")
    submit = SubmitField()

```

1.6 Авторизація користувачів

Для авторизації користувача Flask-Login надає функцію *login_user()*. Вона приймає об'єкт користувача. У разі успіху повертає *True* і встановлює сесію. В іншому випадку - *False*. За замовчуванням сесія, встановлена *login_user()*, закінчується при закритті браузера. Щоб дозволити користувачам залишатися авторизованими на

довше, потрібно передати *remember=True* функції *login_user()* під час авторизації користувача. Відкриємо *main.py*, щоб змінити функцію подання *login()*:

```
#...
from forms import ContactForm, LoginForm
#...
from flask_login import LoginManager, UserMixin, login_required, login_user,
current_user
#...
@app.route('/login/', methods=['post', 'get'])
def login():
    form = LoginForm()
    if form.validate_on_submit():
        user = db.session.query(User).filter(User.username ==
form.username.data).first()
        if user and user.check_password(form.password.data):
            login_user(user, remember=form.remember.data)
            return redirect(url_for('admin'))

        flash("Invalid username/password", 'error')
        return redirect(url_for('login'))
    return render_template('login.html', form=form)
#...
```

Далі потрібно оновити *login.html*, щоб використовувати клас *LoginForm()*. Потрібно додати у файл такі зміни:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Login</title>
</head>
<body>

    {% for category, message in get_flashed_messages(with_categories=true) %}
    <spam class="{{ category }}">{{ message }}</spam>
    {% endfor %}

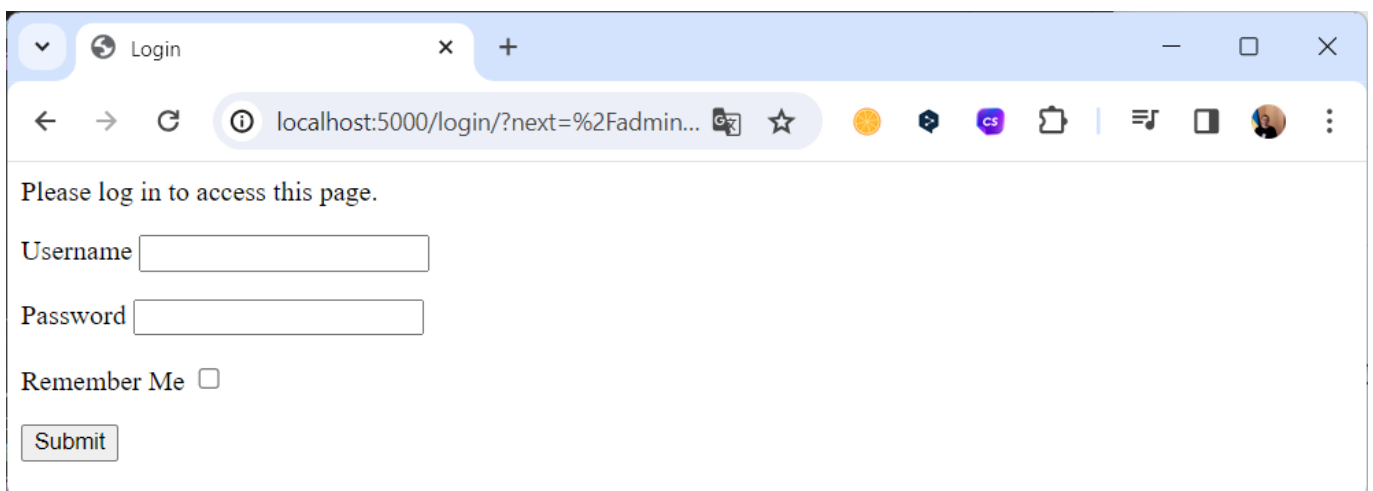
    <form action="" method="post">
    {{ form.csrf_token }}
    <p>
        {{ form.username.label() }}
        {{ form.username() }}
        {% if form.username.errors %}
        {% for error in form.username.errors %}
```

```

        {{ error }}
    {% endfor %}
{% endif %}
</p>
<p>
    {{ form.password.label() }}
    {{ form.password() }}
    {% if form.password.errors %}
    {% for error in form.password.errors %}
        {{ error }}
    {% endfor %}
    {% endif %}
</p>
<p>
    {{ form.remember.label() }}
    {{ form.remember() }}
</p>
<p>
    {{ form.submit() }}
</p>
</form>
</body>
</html>

```

Тепер можна авторизуватися. Якщо зайти <http://localhost:5000/admin>, відбудеться перенаправлення на сторінку авторизації.



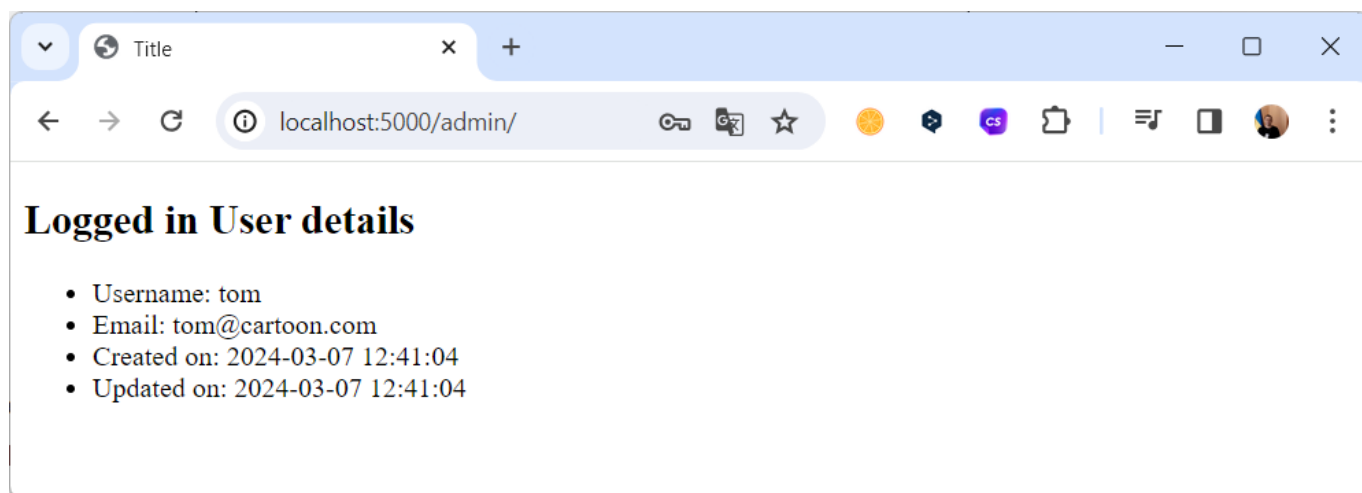
Please log in to access this page.

Username

Password

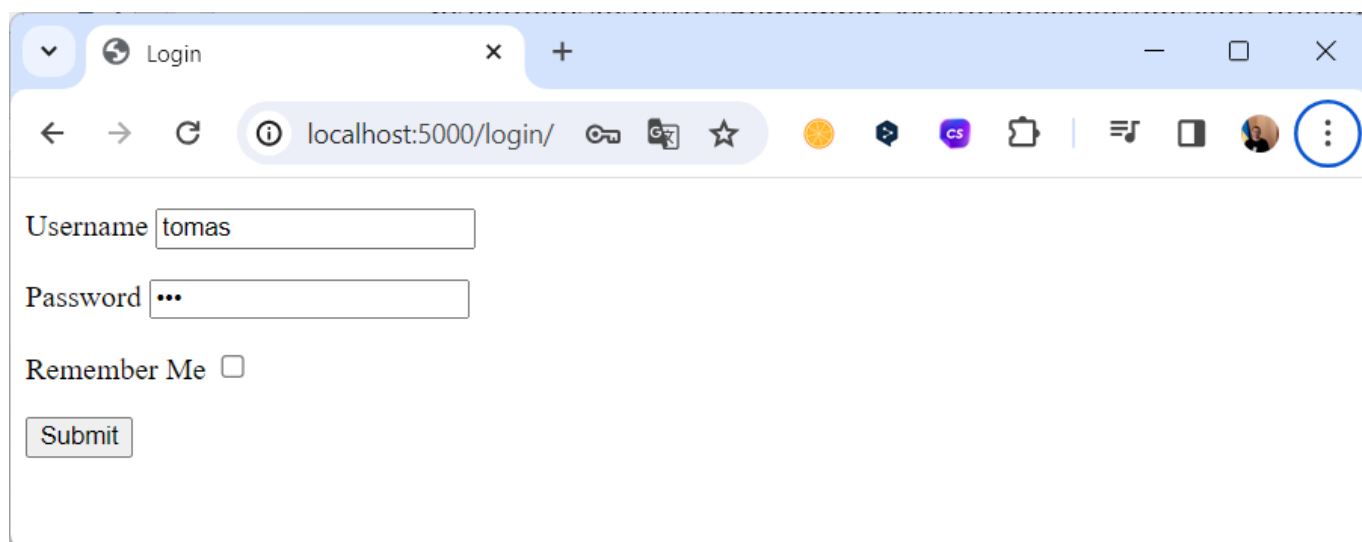
Remember Me ☐

Необхідно ввести коректне ім'я користувача та пароль і натиснути submit. Відбудеться перенаправлення на сторінку адміністратора, яка має мати такий вигляд.



Якщо не клікнути "*Remember Me*" під час авторизації, після закриття браузера сайт вийде з акаунта. Якщо клікнути, то логін залишиться.

Якщо ввести неправильні ім'я користувача або пароль, відбудеться перенаправлення на сторінку авторизації зі спливаючим повідомленням:



The screenshot shows a web browser window with the title 'Login'. The address bar displays 'localhost:5000/login/'. The page content includes an error message 'Invalid username/password' at the top. Below it are two input fields: 'Username' and 'Password'. There is a 'Remember Me' checkbox and a 'Submit' button at the bottom.

1.7 Завершення сеансів користувачів (вихід з акаунтів)

Функція `logout_user()` у Flask-Login завершує сеанс користувача, видаляючи його ідентифікатор із сесії. У файлі `main.py` потрібно додати наступний код під функцією представлення `login()`:

```
#...
from flask_login import LoginManager, UserMixin, login_required, login_user,
current_user, logout_user

#...
@app.route('/logout/')
@login_required
def logout():
    logout_user()
    flash("You have been logged out.")
    return redirect(url_for('login'))
#...
```

Далі необхідно оновити шаблон `admin.html`, щоб додати посилання на маршрут `logout`:

```
<ul>
<li>Username: {{ current_user.username }}</li>
<li>Email: {{ current_user.email }}</li>
<li>Created on: {{ current_user.created_on }}</li>
```

```

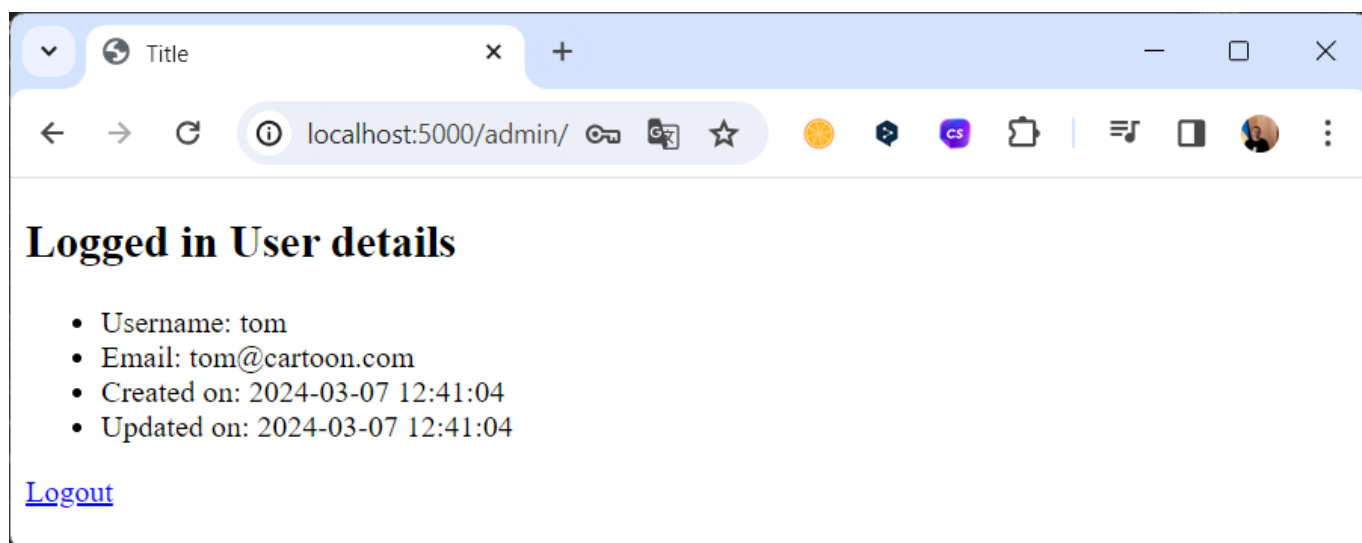
    <li>Updated on: {{ current_user.updated_on }}</li>
</ul>

<p><a href="{{ url_for('logout') }}">Logout</a></p>

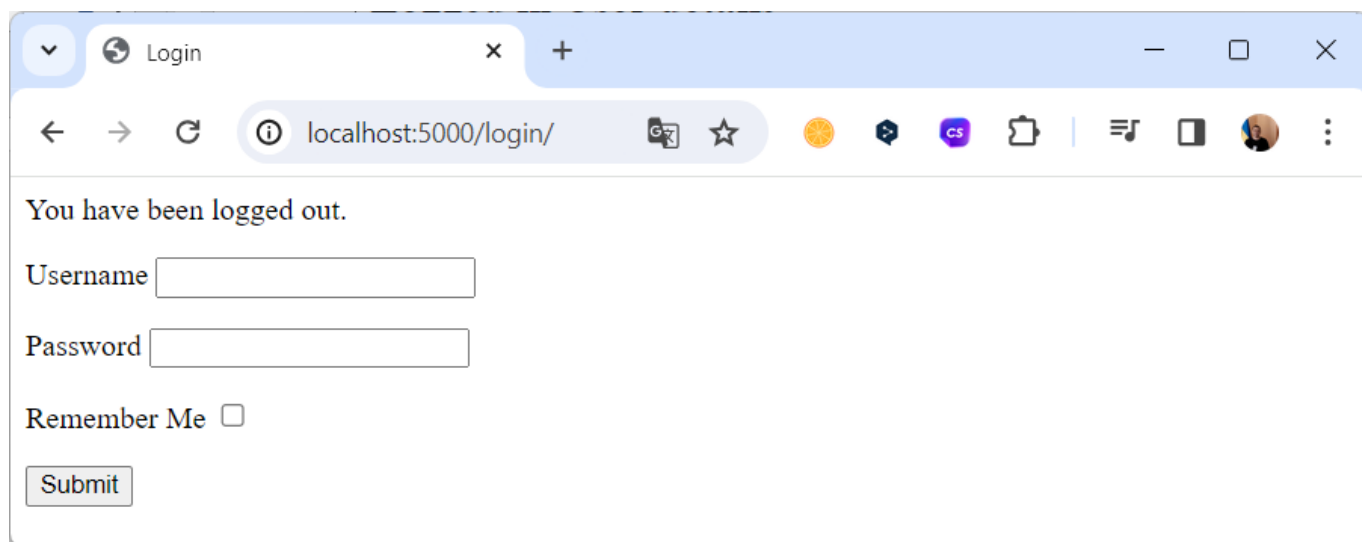
</body>
</html>

```

Якщо зараз зайти на <http://localhost:5000/admin/> (будучи авторизованим), то в нижній частині сторінки має бути посилання для виходу з акаунта.



Якщо її натиснути, відбудеться перенаправлення на сторінку авторизації.



1.8 Фінальні штрихи

Є одна маленька проблема зі сторінкою авторизації. Зараз якщо авторизований користувач зайде на <http://localhost:5000/login/>, то він знову побачить сторінку авторизації. Немає сенсу в демонстрації форми авторизованому користувачеві. Для розв'язання цієї проблеми потрібно додати такі зміни у функцію подання *login()*:

```
#...
@app.route('/login/', methods=['post', 'get'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('admin'))

    form = LoginForm()
    if form.validate_on_submit():
#...
```

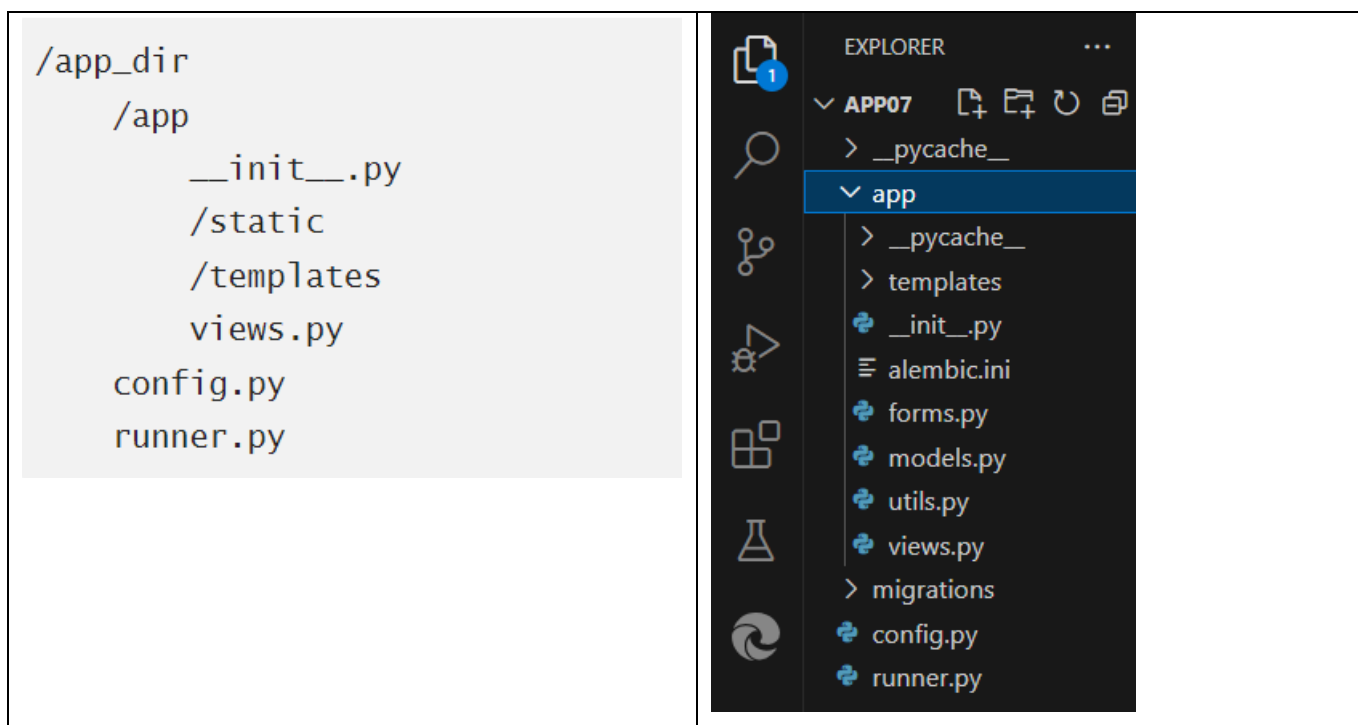
Після цих змін якщо авторизований користувач зайде на сторінку авторизації, його буде перенаправлено на сторінку адміністратора.

2 СТРУКТУРА ТА ЕСКІЗ ДОДАТКА FLASK

До цього моменту весь додаток зберігався в одному файлі *main.py*. Це нормально для маленьких програм, але коли масштаби зростають, ними стає складно керувати. Якщо розбити великий файл на кілька, код у кожному з них стає читабельнішим і передбачуванішим.

У Flask немає жодних обмежень у плані структурування додатків. Проте існують поради (про те, як робити їх модульними).

Ми будемо використовувати таку структуру програми.



Нижче опис кожного файлу і папки:

Файл	Опис
<code>app_dir</code>	Коренева папка проєкту
<code>app</code>	Пакет Python із файлами подання, шаблонами та статичними файлами
<code>__init__.py</code>	Цей файл повідомляє Python, що папка <code>app</code> - пакет Python
<code>static</code>	Папка зі статичними файлами проєкту
<code>templates</code>	Папка з шаблонами
<code>views.py</code>	Маршрути та функції подання
<code>config.py</code>	Налаштування програми
<code>runner.py</code>	Точка входу програми

Частина теми, що залишилася, буде присвячена перетворенню проєкту до такої структури. Почнемо зі створення *config.py*.

2.1 Налаштування на основі класів

Проєкт зі створення ПЗ зазвичай працює в трьох різних середовищах:

- Розробка
- Тестування
- Робочий режим

Під час розвитку проєкту знадобиться визначити різні параметри для різних середовищ. Деякі також залишатимуться незмінними незалежно від середовища. Впровадити таку систему конфігурації можна за допомогою класів.

Почати варто з визначення налаштувань за замовчуванням у базовому класі і тільки потім - створювати класи для окремих середовищ, які успадковуватимуть параметри з базового. Спеціальні класи можуть перезаписувати або доповнювати налаштування, необхідні для конкретного середовища.

Створимо файл *config.py* всередині папки *app07* і додамо такий код:

```
import os

app_dir = os.path.abspath(os.path.dirname(__file__))

class BaseConfig:
    SECRET_KEY = os.environ.get('SECRET_KEY') or 'A SECRET KEY'
    SQLALCHEMY_TRACK_MODIFICATIONS = False

class DevelopmentConfig(BaseConfig):
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = os.environ.get('DEVELOPMENT_DATABASE_URI') or \
        'mysql+pymysql://admin:masterkey@localhost/app05_db'

class TestingConfig(BaseConfig):
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = os.environ.get('TESTING_DATABASE_URI') or \
        'mysql+pymysql://admin:masterkey@localhost/app05_db'

class ProductionConfig(BaseConfig):
    DEBUG = False
    SQLALCHEMY_DATABASE_URI = os.environ.get('PRODUCTION_DATABASE_URI') or \
        'mysql+pymysql://admin:masterkey@localhost/app05_db'
```

Варто звернути увагу, що в цьому коді значення деяких налаштувань вперше беруться у змінних середовища. Також тут є деякі значення за замовчуванням, якщо такі для середовищ не будуть вказані. Цей метод особливо корисний, коли є конфіденційна інформація, і її не хочеться вписувати безпосередньо в додаток.

Зчитувати налаштування з класу буде метод *from_object()*:

```
app.config.from_object('config.Create')
```

2.2 Створення пакета додатка

У папці *app07* потрібно створити нову папку під назвою *app* і перемістити всі файли і папки в неї (за винятком *env* і *migrations*, а також створеного щойно файлу *config.py*). У середині папки *app* потрібно створити файл *__init__.py* з таким кодом:

```
from flask import Flask
from flask_migrate import Migrate
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager
import os, config

# створення екземпляра програми
app = Flask(__name__)
app.config.from_object(os.environ.get('FLASK_ENV') or 'config.DevelopmentConfig')

# ініціалізуємо розширення
db = SQLAlchemy(app)
migrate = Migrate(app, db)
login_manager = LoginManager(app)
login_manager.login_view = 'login'

from . import views
```

__init__.py створює екземпляр додатка і запускає розширення. Якщо змінна середовища *FLASK_ENV* не задана, застосунок запуститься в режимі налагодження (тобто, *app.debug = True*). Щоб перевести додаток у робочий режим, потрібно встановити для змінної середовища *FLASK_ENV* значення *config.ProductionConfig*.

Після запуску розширень інструкція *import* на 21 рядку імпортує всі функції подання. Це потрібно, щоб під'єднати екземпляр застосунку до цих функцій, інакше *Flask* не буде про них знати.

Необхідно перейменувати файл *main.py* на *views.py* і оновити його так, щоб він містив лише маршрути та функції подання. Це повний код оновленого файлу *views.py*.

```
from . import app
from flask import render_template, request, redirect, url_for, flash, make_response, session
from flask_login import login_required, login_user, current_user, logout_user
from .models import User, Post, Category, Feedback, db
from .forms import ContactForm, LoginForm

@app.route('/')
def index():
    return render_template('index.html', name='Tom & Jerry')

@app.route('/user/<int:user_id>/')
def user_profile(user_id):
    return "Profile page of user #{}".format(user_id)

@app.route('/books/<genre>/')
def books(genre):
    return "All Books in {} category".format(genre)

@app.route('/login/', methods=['post', 'get'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('admin'))
    form = LoginForm()
    if form.validate_on_submit():
        user = db.session.query(User).filter(User.username == form.username.data).first()
        if user and user.check_password(form.password.data):
            login_user(user, remember=form.remember.data)
            return redirect(url_for('admin'))
        flash("Invalid username/password", 'error')
        return redirect(url_for('login'))
    return render_template('login.html', form=form)

@app.route('/logout/')
@login_required
def logout():
    logout_user()
    flash("You have been logged out.")
    return redirect(url_for('login'))
```

```

@app.route('/contact/', methods=['get', 'post'])
def contact():
    form = ContactForm()
    if form.validate_on_submit():
        name = form.name.data
        email = form.email.data
        message = form.message.data

        # логіка БД
        feedback = Feedback(name=name, email=email, message=message)
        db.session.add(feedback)
        db.session.commit()

        flash("Message Received", "success")
        return redirect(url_for('contact'))

    return render_template('contact.html', form=form)

@app.route('/cookie/')
def cookie():
    if not request.cookies.get('foo'):
        res = make_response("Setting a cookie")
        res.set_cookie('foo', 'bar', max_age=60*60*24*365*2)
    else:
        res = make_response("Value of cookie foo is {}".format(request.cookies.get('foo')))
    return res

@app.route('/delete-cookie/')
def delete_cookie():
    res = make_response("Cookie Removed")
    res.set_cookie('foo', 'bar', max_age=0)
    return res

@app.route('/article', methods=['POST', 'GET'])
def article():
    if request.method == 'POST':
        res = make_response("")
        res.set_cookie("font", request.form.get('font'), 60*60*24*15)
        res.headers['location'] = url_for('article')
        return res, 302

    return render_template('article.html')

@app.route('/visits-counter/')
def visits():
    if 'visits' in session:
        session['visits'] = session.get('visits') + 1

```

```

    else:
        session['visits'] = 1
    return "Total visits: {}".format(session.get('visits'))

@app.route('/delete-visits/')
def delete_visits():
    session.pop('visits', None) # удаление посещений
    return 'Visits deleted'

@app.route('/session/')
def updating_session():
    res = str(session.items())

    cart_item = {'pineapples': '10', 'apples': '20', 'mangoes': '30'}
    if 'cart_item' in session:
        session['cart_item']['pineapples'] = '100'
        session.modified = True
    else:
        session['cart_item'] = cart_item
    return res

@app.route('/admin/')
@login_required
def admin():
    return render_template('admin.html')

```

views.py містить не тільки функції подання.

Сюди у файл *models.py* переміщено код моделей, класів форм *forms.py* та інші функції для відповідних файлів:

```

from app import db, login_manager
from datetime import datetime
from flask_login import (LoginManager, UserMixin, login_required, login_user,
current_user, logout_user)
from werkzeug.security import generate_password_hash, check_password_hash

class Category(db.Model):
    __tablename__ = 'categories'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(255), nullable=False, unique=True)
    slug = db.Column(db.String(255), nullable=False, unique=True)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    posts = db.relationship('Post', backref='category', cascade='all,delete-orphan')

```

```

    def __repr__(self):
        return "<{}:{}]>".format(self.id, self.name)

post_tags = db.Table('post_tags',
    db.Column('post_id', db.Integer, db.ForeignKey('posts.id')),
    db.Column('tag_id', db.Integer, db.ForeignKey('tags.id'))
)

class Post(db.Model):
    __tablename__ = 'posts'
    id = db.Column(db.Integer(), primary_key=True)
    title = db.Column(db.String(255), nullable=False)
    slug = db.Column(db.String(255), nullable=False)
    content = db.Column(db.Text(), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    updated_on = db.Column(db.DateTime(), default=datetime.utcnow) #,
onupdate=datetime.utcnow)
    category_id = db.Column(db.Integer(), db.ForeignKey('categories.id'))

    def __repr__(self):
        return "<{}:{}]>".format(self.id, self.title[:10])

class Tag(db.Model):
    __tablename__ = 'tags'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(255), nullable=False)
    slug = db.Column(db.String(255), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    posts = db.relationship('Post', secondary=post_tags, backref='tags')
    def __repr__(self):
        return "<{}:{}]>".format(self.id, self.name)

class Feedback(db.Model):
    __tablename__ = 'feedbacks'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(1000), nullable=False)
    email = db.Column(db.String(100), nullable=False)
    message = db.Column(db.Text(), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)

    def __repr__(self):
        return "<{}:{}]>".format(self.id, self.name)

class Employee(db.Model):
    __tablename__ = 'employees'
    id = db.Column(db.Integer(), primary_key=True)

```

```

name = db.Column(db.String(255), nullable=False)
designation = db.Column(db.String(255), nullable=False)
doj = db.Column(db.Date(), nullable=False)

@login_manager.user_loader
def load_user(user_id):
    return db.session.query(User).get(user_id)

class User(db.Model, UserMixin):
    __tablename__ = 'users'
    id = db.Column(db.Integer(), primary_key=True)
    name = db.Column(db.String(100))
    username = db.Column(db.String(50), nullable=False, unique=True)
    email = db.Column(db.String(100), nullable=False, unique=True)
    password_hash = db.Column(db.String(100), nullable=False)
    created_on = db.Column(db.DateTime(), default=datetime.utcnow)
    updated_on = db.Column(db.DateTime(), default=datetime.utcnow,
onupdate=datetime.utcnow)

    def __repr__(self):
        return "<{}:{}]>".format(self.id, self.username)

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

```

Файл *forms.py*

```

from flask_wtf import FlaskForm
from wtforms import Form, ValidationError
from wtforms import StringField, SubmitField, TextAreaField, BooleanField
from wtforms.validators import DataRequired, Email

class ContactForm(FlaskForm):
    name = StringField("Name: ", validators=[DataRequired()])
    email = StringField("Email: ", validators=[Email()])
    message = TextAreaField("Message", validators=[DataRequired()])
    submit = SubmitField()

class LoginForm(FlaskForm):
    username = StringField("Username", validators=[DataRequired()])
    password = StringField("Password", validators=[DataRequired()])

```

```
remember = BooleanField("Remember Me")
submit = SubmitField()
```

Файл *utils.py*

```
from . import mail, db
from flask import render_template
from threading import Thread
from app import app
from flask_mail import Message

def async_send_mail(app, msg):
    with app.app_context():
        mail.send(msg)

def send_mail(subject, recipient, template, **kwargs):
    msg = Message(subject,
sender=app.config['MAIL_DEFAULT_SENDER'], recipients=[recipient])
    msg.html = render_template(template, **kwargs)
    thrd = Thread(target=async_send_mail, args=[app, msg])
    thrd.start()
    return thrd
```

Файл *runner.py*

Нарешті, для запуску додатка потрібно додати такий код у файл *runner.py*:

```
from app import app

if __name__ == "__main__":
    app.run()
```

runner.py - це точка входу проєкту. Метод *run()* буде викликано для запуску сервера.

2.3 Порядок виконання

У цій темі було створено чимало файлів, і досить легко заплутатися в тому, за що відповідає кожен із них, а також у порядку, в якому вони запускаються. Цей розділ створено для роз'яснення всього процесу.

Усе починається з виконання файлу *runner.py*. Другий рядок у файлі *runner.py* імпортує *app* з пакета *app*. Коли інтерпретатор Python доходить до цього рядка, управління програмою передається файлу *__init__.py*, який у цей момент починає виконуватися. На 7 рядку *__init__.py* імпортує модуль *config*, який передає управління *config.py*. Коли виконання *config.py* завершується, керування знову повертається до *__init__.py*. На 21 рядку *__init__.py* імпортує модуль *views*, який передає керування *views.py*. Перший рядок *views.py* знову імпортує екземпляр додатка *app* з пакета *app*. Примірник додатка *app* вже в пам'яті, тому знову його не буде імпортовано. У рядках 4, 5 і 6 *views.py* імпортує моделі, форми та функцію *send_mail* і тимчасово передає управління відповідним файлам. Коли виконання *views.py* завершується, керування програмою повертається до *__init__.py*. Це завершує виконання *__init__.py*. Керування повертається до *runner.py* і починається виконання інструкції в рядку 3.

Третій рядок *runner.py* імпортує класи, визначені в модулі *models.py*. Оскільки моделі вже доступні у файлі *views.py*, файл *models.py* не буде виконуватися.

Оскільки *runner.py* працює як основний модуль, умову в 3 рядку виконано, і *run()* запускає додаток.

2.4 Запуск проєкту

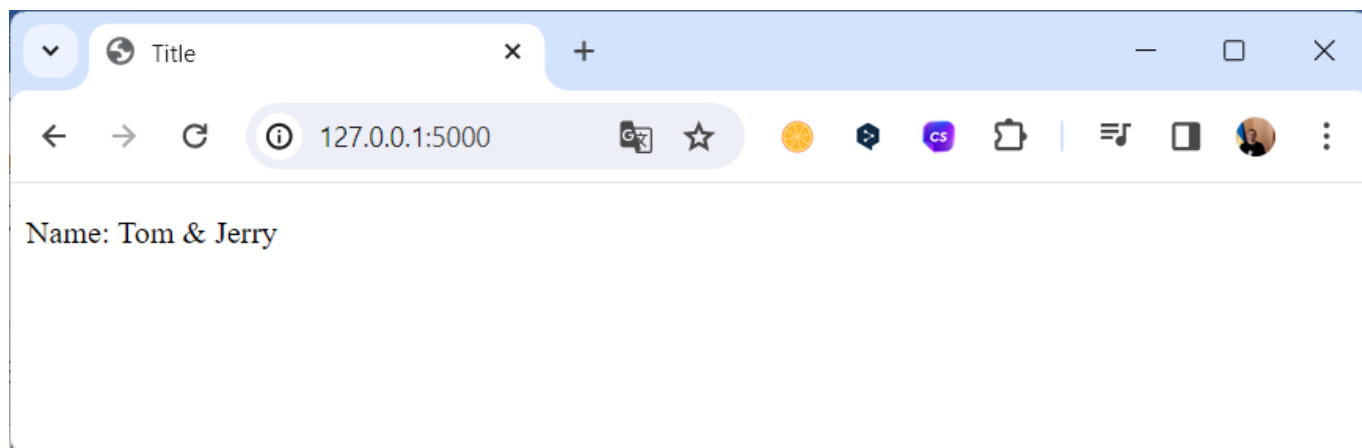
Тепер можна запускати проєкт. У терміналі для запуску сервера потрібно ввести таку команду.

```
python runner.py
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Flask\app07> python runner.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 399-956-126
```

Якщо змінна середовища `FLASK_ENV` не встановлена, попередня команда запустить додаток у режимі налагодження. Якщо зайти на <https://127.0.0.1:5000/>, відкриється домашня сторінка з таким змістом:



Також необхідно перевірити інші сторінки додатка, щоб переконатися, що все працює.

Проєкт тепер у кращій формі. Його елементи організовані більш логічно. Використаний тут підхід підійде для маленьких і середніх за масштабом проєктів. Проте у Flask є ще кілька козирів для тих, хто хоче бути ще продуктивнішим.